Evaluating ALS and Content-Based recommendation systems in temporally dependent news articles

Tiziano Wei Cheng Xie

84563A

20 March 2026

"I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content."

Abstract

The report analyzes the differences of the two principal types of recommendation systems (*Collaborative Filtering(ALS) and Content-Based filtering(CB)*) applied to the NYT articles and comments 2020 dataset, implemented in a context of big data using PySpark. The two systems showed different performance in a temporal context, simulating the real world scenario of recommendation of articles based off historical data. The results indicate that ALS performs better when sufficient interaction data is available, whereas CB is more robust in "cold-start" scenarios.

1 Introduction

With the thousands of articles published daily, the ability to provide personalized recommendations is essential for user engagement and a difficult problem to resolve. This project explores the implementation and evaluation of two recommendation models Collaborative Filtering (PySpark's mllib ALS) and a custom Content-Based (CB implemented from scratch) to a massive dataset of New York Times interactions from 2020.

1.1 Collaborative vs. Content-Based Filtering

Recommendation systems generally fall into two categories:

- **Collaborative Filtering:** an algorithm that identifies patterns by analyzing the collective behavior of users. In this project, we utilize **Alternating Least Squares (ALS)**, which factorizes the user-item utility matrix into latent factors (UV decomposition). It captures hidden similarities (latent factors) between users who share common interests, regardless of the article content. It is, therefore very dependent on user-item interaction data
- **Content-Based Filtering:** this approach focuses on the features of the items themselves. By building a profile for each article (item profile) based on metadata (headlines and keywords) and a corresponding profile for each user (user profile) based on their comments history, the system recommends items similar to those a user liked in the past. The profiles represents the charateristics of an article (item profile) or the preferences of a user (user_profile). The similarity (prediction score) between the two profiles (vectors) is calculated with a similarity function, in this project the cosine similarity was used.

1.2 The Cold Start Problem

A challenge in recommendation systems is the *cold-start* problem, which can be:

- **User cold-start:** a new user joins the platform, the system has no historical data to generate recommendations.
- **Item cold-start:** a new article is published, it has no interactions (comments).

Collaborative filtering models like ALS are particularly susceptible to the item cold-start problem, since they rely on interaction history, they are "blind" to new articles until a sufficient number of users have interacted with them. Instead, Content-Based systems surpass this by utilizing article metadata, allowing for immediate recommendations as soon as an article is published (only if it's possible to recover the necessary features from said article). This project specifically examines how these two models behave in a temporal context where the objective is to predict the newer items of the dataset from older training data.

2 Dataset and Preprocessing

2.1 The Dataset

The dataset used for the project is the New York Times Articles & Comments (2020) version 6 (last access date 20 March 2026) [2]from Kaggle.

The dataset consists of all articles published by the NYT in 2020 and the user comments associated with them. The project uses specifically the `nyt-comments-2020.csv`, `nyt-articles-2020.csv`, `nyt-comments-part0.csv` and `nyt-comments-part1.csv`.

2.2 Data Organization

For faster testing, an option was introduced inside the project to utilize a subset of the `nyt-comments-2020.csv`, which is simply the union of the `nyt-comments-part0.csv` and `nyt-comments-part1.csv` data.

The data was read using PySpark's `SparkSession.read.csv()` function, which converts the csv into a relational table in the DataFrame API of PySpark, the project however utilises the RDD API to handle the data at a Map Reduce level, so the resulting dataframes were converted to RDDs right after selecting the columns needed:

- **Comments:** contains `userID` (integer), `articleID` (string of the article link) and `create_date` (datetime), the date of the creation of the comment. Which is essentially our **utility matrix** (a binary one, where the presence of a user comment on an article indicates the interest on said article)
- **Articles**: contains `articleID` (string of te article link), `headline` (string) and `keywords` (list of strings, already provided by the dataset, which are then used by the CB approach),

2.3 Data Preprocessing

A series of filters and remappings were applied to the data:

- **Integer articleID mapping**: to use PySpark's ALS implementation, the articleIDs were mapped to a unique integer ID using a broadcasted map, created applying the `zipWithIndex` method on the RDD of the distinct article string IDs.
- **Filtering of comments based on date**: for each user, it was retained only the oldest comment on each article the user has commented on, this stems from the idea that if a user commented multiple times on an article, it doesn't indicate a major interest on said article, but instead it's due to conversations with other users (e.g replies).
- **Filtering users based on comments count**: to reduce the problem of user based cold-start, the users who left less than 10 comments were dropped.

2.4 Data Partitioning: Temporal vs Random Split

To evaluate the recommendation models, the partitioning of the utility matrix was designed with a `temporal` flag to actuate two distinct partitioning strategies, dividing the data into training (60%), validation (20%), and test (20%) sets.

- **Random Split:** when the `temporal` flag is disabled, the system utilizes PySpark's `randomSplit([0.6, 0.2, 0.2], seed=0)` method. While this provides a uniform distribution of the dataset, it also creates a problem in causality. Specifically, it allows the model to train on comments from December to predict a user's behavior in January, meaning prediction of a user's taste based on future data.
- **User-Centric Temporal Split:** to alleviate the random split problem (but not completely eliminate) and to simulate a realistic environment, with the temporal flag set to true, we utilize a custom temporal split. The interactions were grouped by `userID` and sorted chronologically. For each individual user, the first 60% of their interactions were assigned to the training set, the next 20% to validation (used for hyperparameter tuning), and the final 20% to the test set.

By partitioning at the user level rather than implementing a global date cutoff, there is, yes, still a certain amount of comments that "leak" into the "past" from the "future", but this is way less than just performing a random split, also by doing so we guarantee that every user in the evaluation phase has a sufficient historical profile (training data) for the ALS algorithm to construct a valid latent representation. And the same applies to the items (articles). If we were to cutoff at a fixed date, we wouldn't have any training data on the interactions with the test items, which causes a complete item based cold-start, which can be mitigated with CB, but there would not be a fair comparison against ALS. In the user centric approach, the leakage of future comments helps to alleviate the item cold-start but still keep the overall temporal logic underneath.

3 Method and Implementation

3.1 Collaborative Filtering (ALS) Implementation

The collaborative filtering system was implemented using the Alternating Least Squares (ALS) algorithm provided by the `pyspark.mllib.recommendation` library.

3.1.1 Model Training via trainImplicit

Since the NYT dataset provides comments rather than explicit user ratings (like 1 to 5 stars), the interactions must be treated as implicit feedback. We utilized the `ALS.trainImplicit()` method , which assumes that the presence of a comment indicates a positive preference, while the absence of a comment

represents lower confidence rather than a negative preference [1, 3]. The model takes the training RDD (consisting of `userID`, mapped integer `articleID`, and a binary value, which in this case is always set to 1.0 since we don't represent the interactions that didn't happen).

3.1.2 Optimized Prediction and Hyperparameter Tuning

While the model training was handled by MLlib, the prediction and evaluation pipeline was custom-built instead of using PySparks' `predictAll()` or `recommendProductsToUsers()` to maximize distributed efficiency

1. Broadcast-Based Matrix Multiplication: Instead of joining the user and item latent factor RDDs, we collected the complete item feature matrix (V) to the driver node (very manageable since our ranks can reach at maximum 64). This matrix, alongside an ID-to-index mapping, was then distributed to all worker nodes using Spark's `SparkContext.broadcast()`. For each user, the recommendation scores for all candidate articles were computed simultaneously using the highly optimized NumPy matrix multiplication:

```
scores = item_rows @ u_lat_vec
```

2. Linear-Time Ranking: to ensure the system only recommended articles not seen in training data, the seen articles were masked by overriding their predicted scores with $-\infty$. Subsequently, we utilized `np.argpartition`(a partial sorting algorithm) to extract the indices of the Top-K items in $O(n)$ linear time. The resulting Top-K set was intersected with the user's test set to calculate the **Precision@10** metric.

3. Grid Search: to optimize the model, a grid search was performed over the validation set.

- **Rank (latent dimension):** evaluated over `[16, 32, 64]` for the full dataset (and `[8, 16, 32]` for rapid prototyping on partial data).
- **Alpha (confidence):** evaluated over `[1.0, 10.0, 40.0]` to tune the weight of the implicit feedback.

The model was trained using a fixed random seed (`seed = 67`), `iterations = 10`, and regularization parameter `lambda_ = 0.1`. The configuration with the highest mean Precision@10 was then selected as the final model.

3.2 Content-Based Filtering Implementation

The Content-Based (CB) system was implemented from scratch using the PySpark RDD API. Using MapReduce paradigm to build a feature-engineered recommendation engine based on article's keywords.

3.2.1 Feature Engineering and Item Profiling

Article features were extracted by parsing the `keywords` field. The raw text was cleaned, tokenized, and transformed into an **item profile** using Python's

`Counter` collection, which maps each token to its raw frequency within the article (for the item profiles, since we're using the keywords already given by the dataset, the frequency is generally 1 for each keyword in the profile). The `Counter` proved very useful when calculating the `user profiles`

To prepare the item profiles for similarity scoring, we applied an $L2$ Normalization (Euclidean norm) to each item vector. By dividing each term frequency by the square root of the sum of squared frequencies, we ensured that all item vectors have a magnitude of 1. These normalized profiles were then collected into a master Python dictionary and distributed to the cluster via a Broadcast Variable

3.2.2 User Profile Construction

To construct the user profiles, a distributed join between the `train_rdd` and the unnormalized `item_profile_rdd` was performed. This mapped every interaction to its underlying article tokens (articleID, (Counter(item_profile), userID)).

Using a `reduceByKey` transformation, we aggregated these tokens for each user by summing their respective `Counter` dictionaries $(x + y)$. Finally, the resulting aggregate user vectors are L2 normalized, facilitating the cosine similarity calculation.

3.2.3 Distributed Cosine Similarity and Ranking

Because both the user vectors and the broadcasted item vectors were already L2-normalized, the similarity score was computed using a simple dot product, which is mathematically equivalent to the cosine similarity in this case. Indeed:

$$\text{Cosine Similarity}(\vec{u}, \vec{i}) = \frac{\vec{u} \cdot \vec{i}}{\|\vec{u}\|\|\vec{i}\|}$$

Both the user profile $\vec{u}$ and the item profile $\vec{i}$ are $L2$-normalized, their magnitudes are exactly 1 ($\|\vec{u}\| = 1$ and $\|\vec{i}\| = 1$). Therefore:

$$\begin{aligned}\text{Cosine Similarity}(\vec{u}, \vec{i}) &= \frac{\vec{u} \cdot \vec{i}}{1 \cdot 1} \\ &= \vec{u} \cdot \vec{i}\end{aligned}$$

Since the dot product between two vectors

$$\vec{u} \cdot \vec{i} = \sum_{k \in T} u_k \cdot i_k$$

Unlike the latent factors of ALS, which were dense, the item and user profiles are indeed very sparse (represented as dictionaries), $u_k = 0$ for any token not in the user's profile, and $i_k = 0$ for any token not in the article. Thus, the product $u_k \cdot i_k$ is only non-zero when token k exists in *both* vectors ($U \cap I$). this allows us to compute the dot product by summing only over the components that are non zero in both profiles: The computation therefore simplifies entirely to the intersection :

$$\vec{u} \cdot \vec{i} = \sum_{k \in (U \cap I)} u_k \cdot i_k$$

This allows us to filter out first the articles sharing no tokens with the user profile.

Articles present in the user's training set are explicitly skipped to enforce novelty. Once obtained the prediction scores for each item, the top-k set is identified thanks to the `np.argpartition` method, as seen in ALS. The Precision@10 is computed and for the evaluation averaged over all the users.

4 Experiments and Discussion

4.1 Optimal Hyperparameters

Before the final evaluation on the test set, the Collaborative Filtering (ALS) model underwent a grid search optimization using the 20% validation set. The regularization parameter was held constant at `lambda_ = 0.1` with 10 iterations. The grid search yielded the following optimal configurations for each partitioning strategy (full dataset):

- **Temporal split:** `rank` = 64, `alpha` = 1.0
- **Random split:** `rank` = 32, `alpha` = 10.0

We can see that a higher rank was needed when ALS was struggling for the temporal split, but a lower alpha was needed, which means the model treated the training data with less confidence. *(Note: The Content-Based model does not require hyperparameter tuning in this implementation, but the choice of the features could have been tuned to achieve better scores, but that was outside the scope of this project).*

4.2 Results

The primary metric used to evaluate the models was **Precision@10**, which measures the proportion of the top 10 recommended articles that the user actually interacted with in the test set. To have a baseline to compare to for both models, a simple popularity based model was introduced, which only outputs the top 10 most popular articles.

The models are evaluated under the two partitioning modes: the user-centric temporal split and a random split.

Table 1: Mean Precision@10 Comparison: Temporal vs Random Split

Recommendation Model	**Temporal Split**	**Random Split**
Collaborative Filtering (ALS)	0.01968%	**2.73481%**
Content-Based Filtering (CB)	**0.35192%**	0.65882%
Popularity Baseline	0.01225%	1.07838%

4.3 Discussion

The results highlight the fundamental trade-offs between Collaborative and Content-Based filtering in a temporal news environment.

ALS: barely surpassed the Popularity baseline model at the temporal split and worse by a factor of x18 compared to CB, but has the best precision in random split (twice the popularity baseline and x5 the CB precision). Assuming the user has a sufficient interaction history and the items have been seen by others, ALS can capture complex, latent social patterns that keyword matching cannot. However, ALS is inherently limited by the *item cold-start* problem (also by the user cold-start, but the problem in the project was dealt with artificially dropping the users that didn't have enough comments in the whole dataset). In the temporal split, articles published later have seen very few training interactions, making it hard for ALS to position them accurately in the latent space.

CB: the custom Content-Based implementation, while definitely not the best precision scores were achieved by the model, it still shows significant robustness compared to ALS in item cold-start. Since the system relies entirely on the `keywords` metadata, an article is immediately recommendable the moment it is published, regardless of how many other users have commented on it. As mentioned in the results notes, more features (publication date, keywords from the stack of headline + abstract + keywords, category etc.) could have been used to achieve better precisions, but the important result, even in such a simple model, resulted in better scores against ALS.

5 Conclusion

This project successfully implemented and compared the two major recommendation paradigms on a massive real-world dataset using PySpark. The experiments demonstrate that ALS struggles in an item cold-start context, while CB manages to remain robust. But in a random split of the data, ALS prevails. Thus we can think of actuating a hybrid solution, where ALS is used when there are enough information on the items, and CB for the cold-start scenarios.

References

[1] Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. http://yifanhu.net/PUB/cf.pdf

[2] Awd, Benjamin. (2020). *New York Times Articles & Comments (2020)*. Kaggle. Retrieved March 20, 2026, from https://www.kaggle.com/datasets/benjaminawd/new-york-times-articles-comments-2020/data

[3] Apache Software Foundation. (n.d.). *Collaborative Filtering - RDD-based API - Spark Documentation*. Retrieved March 20, 2026, from https://spark.apache.org/docs/latest/mllib-collaborative-filtering.html